

Modul Praktikum 3: Structures and Union and, function and recursion dalam Pemrograman C

- LO 2: Analyze programs to solve problems using the C language and AI problem-solving sessions related to the topics.
- LO 3: Evaluate algorithms, syntax, and functions for problem-solving using the C language and AI problem-solving sessions related to the topics

Sesi Kolaborasi AI: Analisis Data Modeling dan Evaluasi Algoritma

Konsep Inti: Penggunaan AI dalam modul ini berfokus pada **LO 2 (Menganalisis program untuk memecahkan masalah)** dan **LO 3 (Evaluasi jenis algoritma)**. Mahasiswa akan menggunakan AI untuk memahami perbedaan alokasi memori (*struct* vs *union*) dan memvalidasi kebenaran logika algoritma yang intensif, seperti *sorting* dan *searching*.

Komponen HAICOLA	Fokus Pembelajaran	Aktivitas Kolaborasi AI & Refleksi
H-Holistic Abstraction	Pemodelan Data Kompleks: Pahami perbedaan mendasar antara struct (menyimpan semua anggota secara terpisah) dan union (berbagi ruang memori yang sama).	Tugas AI: Tanyakan kepada AI: "Jika sebuah struct memiliki int i (4 byte) dan float f (4 byte), berapakah ukuran totalnya? Bandingkan dengan union yang memiliki anggota yang sama."
A-AI-Assisted Debugging (Structure)	Akses Anggota: Memperkuat pemahaman tentang akses anggota <i>struct</i> menggunakan operator titik (.) (seperti pada Latihan 1).	Tugas AI: Sengaja buat <i>syntax error</i> dengan mencoba mengakses anggota <i>struct</i> menggunakan <i>pointer</i> tetapi menggunakan operator titik (.). Permintaan: "Program saya untuk Latihan 1 gagal saat mencoba mengakses nama siswa. Jelaskan kesalahan saya dan kapan saya harus menggunakan operator panah (->)?"
I-Introspection & Correction (Union)	Analisis Integritas Data: Memahami bahwa hanya satu anggota union yang boleh diakses pada satu waktu.	Tugas AI: Ambil Contoh Program Unions . Minta AI untuk menjelaskan apa yang terjadi pada nilai data.i (yang diatur ke 10) setelah nilai data.f (220.5) diatur. Refleksi: Tulis

		kesimpulan tentang risiko <i>data corruption</i> pada union.
C-Concept Expansion (Sorting)	Pelacakan Logika Algoritma: Fokus pada <i>traceability</i> algoritma <i>sorting</i> (Bubble Sort, Selection Sort, Insertion Sort), yang sangat rentan terhadap kesalahan logika <i>loop</i> .	Tugas AI: Berikan kode Bubble Sort . Permintaan: "Jelaskan langkah demi langkah proses <i>swap</i> dan perbandingan pada <i>array</i> {5, 1, 4} untuk iterasi luar pertama. Berapa perbandingan yang terjadi?"
O-Operator Evaluation (Searching)	Kesesuaian Algoritma: Mengevaluasi jenis <i>searching</i> yang paling efisien berdasarkan kondisi data (terurut vs. tidak terurut).	Tugas AI: Berikan skenario: "Jika data saya tidak terurut (seperti Latihan Mandiri 2), apakah Binary Search adalah pilihan terbaik? Jelaskan mengapa Binary Search mensyaratkan data harus terurut dan bagaimana dampaknya pada kompleksitas waktu."
LA-Learned Application	Pengujian Kombinasi Algoritma: Menerapkan urutan yang benar: Sorting harus selalu mendahului Binary Search (seperti pada Latihan 3).	Final Check AI: Minta AI untuk membandingkan implementasi Anda di Latihan 3 (Bubble Sort & Binary Search) dengan <i>template</i> optimal. Permintaan: "Jika saya mengganti Bubble Sort dengan Insertion Sort di Latihan 3, bagaimana dampaknya pada performa total program (Waktu pengurutan + Waktu pencarian)?"

Poin Penting untuk Kolaborasi AI yang Efektif

1. **Analisis Memori (struct & union):** Gunakan AI untuk **memverifikasi ukuran memori** dari *struct* dan *union* yang Anda definisikan. Ini adalah indikator langsung seberapa baik Anda memahami alokasi memori di C.
2. **Validasi Algoritma:** Saat mengimplementasikan *sorting*, **jangan ragu** untuk meminta AI melacak beberapa langkah awal algoritma Anda. Ini adalah cara tercepat untuk menemukan kesalahan **Off-by-One Errors** atau kesalahan logika pertukaran (*swap*).
3. **Prioritas Searching:** Ingat aturan emas: **Binary Search** hanya berfungsi pada data **terurut**. Minta AI untuk mengingatkan Anda tentang langkah-langkah yang diperlukan sebelum menggunakan Binary Search (LO 3).

Pendahuluan

Modul ini akan membahas dasar-dasar structures dan unions dalam bahasa pemrograman C serta algoritma searching dan sorting. Topik ini sangat penting untuk pengelolaan data yang lebih kompleks dan efisien dalam pengembangan perangkat lunak.

Tujuan

1. Memahami penggunaan struktur (structure) dan union (union).
2. Mampu mengimplementasikan dan menggunakan struktur dan union dalam program C.
3. Memahami dan mengimplementasikan algoritma searching (linear dan binary search).
4. Memahami dan mengimplementasikan algoritma sorting (bubble sort, selection sort, dan insertion sort).

Materi

1. Structures

Structures adalah tipe data yang memungkinkan pengelompokan beberapa variabel dengan tipe data yang berbeda.

Deklarasi dan Penggunaan Structures

```
#include <stdio.h>

struct Student {
    char name[50];
    int age;
    float gpa;
};

int main() {
    struct Student student1;

    // Input data
    printf("Enter name: ");
    scanf("%s", student1.name);
    printf("Enter age: ");
    scanf("%d", &student1.age);
    printf("Enter GPA: ");
    scanf("%f", &student1.gpa);

    // Output data
    printf("Name: %s\n", student1.name);
    printf("Age: %d\n", student1.age);
    printf("GPA: %.2f\n", student1.gpa);

    return 0;
}
```

2. Unions

Unions adalah tipe data yang memungkinkan penyimpanan beberapa tipe data dalam memori yang sama.

Deklarasi dan Penggunaan Unions

```
#include <stdio.h>

union Data {
    int i;
    float f;
    char str[20];
};

int main() {
    union Data data;

    data.i = 10;
    printf("data.i : %d\n", data.i);

    data.f = 220.5;
    printf("data.f : %.1f\n", data.f);

    strcpy(data.str, "C Programming");
    printf("data.str : %s\n", data.str);

    return 0;
}
```

3. Searching

Linear Search

```
#include <stdio.h>

int linearSearch(int arr[], int size, int key) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == key) {
            return i;
        }
    }
    return -1;
}

int main() {
    int arr[] = {2, 4, 0, 1, 9};
    int key = 1;
    int size = sizeof(arr) / sizeof(arr[0]);

    int index = linearSearch(arr, size, key);

    if (index != -1) {
        printf("Element found at index: %d\n", index);
    } else {
        printf("Element not found\n");
    }

    return 0;
}
```

Binary Search

```
#include <stdio.h>

int binarySearch(int arr[], int size, int key) {
    int left = 0, right = size - 1;
    while (left <= right) {
```

```

        int mid = left + (right - left) / 2;

        if (arr[mid] == key) {
            return mid;
        }

        if (arr[mid] < key) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int key = 3;
    int size = sizeof(arr) / sizeof(arr[0]);

    int index = binarySearch(arr, size, key);

    if (index != -1) {
        printf("Element found at index: %d\n", index);
    } else {
        printf("Element not found\n");
    }

    return 0;
}

```

4. Sorting

Bubble Sort

```

#include <stdio.h>

void bubbleSort(int arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int size = sizeof(arr) / sizeof(arr[0]);

    bubbleSort(arr, size);

    printf("Sorted array: \n");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}

```

Selection Sort

```
#include <stdio.h>

void selectionSort(int arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < size; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        int temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int size = sizeof(arr) / sizeof(arr[0]);

    selectionSort(arr, size);

    printf("Sorted array: \n");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

Insertion Sort

```
#include <stdio.h>

void insertionSort(int arr[], int size) {
    for (int i = 1; i < size; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

int main() {
    int arr[] = {12, 11, 13, 5, 6};
    int size = sizeof(arr) / sizeof(arr[0]);

    insertionSort(arr, size);

    printf("Sorted array: \n");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

Praktikum

Latihan 1: Menggunakan Structures

Buatlah program yang meminta pengguna untuk memasukkan data beberapa siswa (nama, umur, dan nilai), menyimpannya dalam array of structures, dan kemudian mencetak data tersebut.

```
#include <stdio.h>

struct Student {
    char name[50];
    int age;
    float grade;
};

int main() {
    int n;
    printf("Enter the number of students: ");
    scanf("%d", &n);

    struct Student students[n];

    for (int i = 0; i < n; i++) {
        printf("Enter name, age, and grade for student %d:\n", i + 1);
        scanf("%s %d %f", students[i].name, &students[i].age, &students[i].grade);
    }

    printf("\nStudent Details:\n");
    for (int i = 0; i < n; i++) {
        printf("Name: %s, Age: %d, Grade: %.2f\n", students[i].name, students[i].age,
students[i].grade);
    }

    return 0;
}
```

Latihan 2: Menggunakan Unions

Buatlah program yang menggunakan union untuk menyimpan berbagai jenis data (integer, float, dan string) dan mencetak nilai dari masing-masing data.

```
#include <stdio.h>
#include <string.h>

union Data {
    int i;
    float f;
    char str[20];
};

int main() {
    union Data data;

    data.i = 10;
    printf("data.i : %d\n", data.i);

    data.f = 220.5;
    printf("data.f : %.1f\n", data.f);

    strcpy(data.str, "C Programming");
}
```

```
printf("data.str : %s\n", data.str);

return 0;
}
```

Latihan 3: Implementasi Searching dan Sorting

Buatlah program yang meminta pengguna untuk memasukkan sejumlah angka, kemudian menggunakan bubble sort untuk mengurutkan angka-angka tersebut dan mencari angka tertentu menggunakan binary search.

```
#include <stdio.h>

void bubbleSort(int arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int binarySearch(int arr[], int size, int key) {
    int left = 0, right = size - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == key) {
            return mid;
        }

        if (arr[mid] < key) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter the elements:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    bubbleSort(arr, n);

    printf("Sorted array: \n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    int key;
```

```
printf("Enter the element to search: ");
scanf("%d", &key);

int index = binarySearch(arr, n, key);

if (index != -1) {
    printf("Element found at index: %d\n", index);
} else {
    printf("Element not found\n");
}

return 0;
}
```

LATIHAN MANDIRI

Latihan 1:

Buatlah program yang meminta pengguna untuk memasukkan data beberapa produk (nama, harga, dan stok), menyimpannya dalam array of structures, dan kemudian mencetak data produk dengan harga tertinggi.

Latihan 2:

Buatlah program yang meminta pengguna untuk memasukkan sejumlah angka, kemudian menggunakan insertion sort untuk mengurutkan angka-angka tersebut dan mencari angka tertentu menggunakan linear search.

Sesi Kolaborasi AI: Debugging Modularitas dan Analisis Logika Rekursif

Konsep Inti: Penggunaan AI dalam modul ini berfokus pada **struktur program yang terorganisir** dan **efisiensi algoritma rekursif**. Mahasiswa akan menggunakan AI untuk memvalidasi *function signature* dan menganalisis *flow control* dari proses rekursi, yang secara langsung mendukung **Tujuan: Menyusun program yang modular, terstruktur, dan efisien**.

Komponen HAICOLA	Fokus Pembelajaran	Aktivitas Kolaborasi AI & Refleksi
H-Holistic Abstraction	Konsep Dasar Rekursi: Pahami dua elemen penting dalam setiap fungsi rekursif: Base Case (Kondisi Berhenti) dan Recursive Step (Langkah Rekursif) .	Tugas AI: Minta AI untuk menjelaskan apa yang terjadi pada memori komputer (Stack Overflow) jika <i>Base Case</i> pada fungsi factorial dihilangkan. Refleksi: Tulis definisi Anda sendiri tentang <i>Base Case</i> dan hubungannya dengan pemecahan masalah.

A-AI-Assisted Debugging (Fungsi)	Validasi Function Signature: Fokus pada kesalahan <i>mismatch</i> antara deklarasi (prototype), pemanggilan, dan definisi fungsi (terkait tipe data dan jumlah parameter). (Latihan 1)	Tugas AI: Ubah tipe kembalian fungsi <code>calculateArea</code> dari <code>float</code> menjadi <code>int</code> di definisi, tetapi biarkan <code>float</code> di deklarasi. Permintaan: "Kompiler C saya memberikan peringatan tentang <i>type mismatch</i> di fungsi <code>calculateArea</code> . Tolong tunjukkan di mana deklarasi, pemanggilan, dan definisi saya tidak konsisten."
I-Introspection & Correction (Parameter)	Analisis Pass-by-Value vs. Pass-by-Reference: Pahami bahwa fungsi C secara <i>default</i> menggunakan <i>pass-by-value</i> (kecuali menggunakan <i>pointer</i>). (Latihan Mandiri 1: <code>reverseString</code>)	Tugas AI: Berikan kode Latihan Mandiri 1. Permintaan: "Mengapa fungsi <code>reverseString</code> ini bisa memodifikasi <code>str</code> asli di <code>main</code> , padahal fungsi <code>calculateArea</code> (Latihan 1) tidak? Jelaskan perbedaan Pass-by-Value dan Pass-by-Reference pada konteks <i>array</i> ."
C-Concept Expansion (Rekursi)	Pelacakan Stack Frame: Memvisualisasikan bagaimana komputer melacak panggilan rekursif (Contoh: <code>factorial</code> atau <code>power</code>).	

Poin Penting untuk Kolaborasi AI yang Efektif

1. **Konsistensi adalah Kunci:** Selalu gunakan AI untuk memverifikasi bahwa **Deklarasi**, **Definisi**, dan **Pemanggilan** fungsi Anda konsisten, terutama dalam hal tipe data dan urutan parameter.
2. **Base Case Wajib:** Dalam setiap latihan rekursi, jadikan AI sebagai validator untuk **Kondisi Berhenti**. Tidak adanya *Base Case* adalah *bug* paling fatal dalam rekursi.
3. **Memori dan Kinerja:** Gunakan AI untuk memahami bahwa rekursi, meskipun elegan, seringkali mengonsumsi lebih banyak memori (**Stack**) daripada solusi iteratif (`loop`). Hal ini penting untuk **evaluasi algoritma** (LO 3).

Pendahuluan

Modul ini membahas dasar-dasar fungsi dan rekursi dalam bahasa pemrograman C. Pemahaman tentang fungsi dan rekursi sangat penting untuk menyusun program yang modular, terstruktur, dan efisien. Fungsi memungkinkan kita untuk mengorganisir kode menjadi bagian-bagian yang dapat digunakan kembali, sedangkan rekursi adalah teknik pemrograman di mana sebuah fungsi memanggil dirinya sendiri untuk memecahkan masalah yang lebih kecil.

Tujuan

1. Memahami dasar-dasar penggunaan fungsi dalam bahasa C.
2. Mampu mengimplementasikan berbagai jenis fungsi dalam program C.
3. Memahami konsep rekursi dan penggunaannya dalam algoritma.
4. Mampu mengimplementasikan algoritma rekursif dalam program C.

Materi

1. Fungsi

Fungsi adalah blok kode yang dirancang untuk melakukan tugas tertentu. Penggunaan fungsi mempermudah pembacaan dan pemeliharaan kode.

Deklarasi dan Definisi Fungsi

Deklarasi fungsi memberitahu kompiler tentang nama fungsi, tipe kembalian, dan parameter. Definisi fungsi memberikan implementasi aktual dari fungsi tersebut.

```
#include <stdio.h>

// Deklarasi fungsi
int add(int a, int b);

int main() {
    int sum;
    sum = add(5, 3);
    printf("Sum: %d\n", sum);
    return 0;
}

// Definisi fungsi
int add(int a, int b) {
    return a + b;
}
```

Penjelasan:

- `int add(int a, int b);` adalah deklarasi fungsi `add` yang menerima dua parameter integer dan mengembalikan nilai integer.
- `sum = add(5, 3);` memanggil fungsi `add` dengan argumen 5 dan 3.
- Fungsi `add` dijelaskan setelah `main`, mengembalikan jumlah `a` dan `b`.

Fungsi dengan Nilai Kembalian dan Parameter

Fungsi dapat mengembalikan nilai dan menerima parameter untuk melakukan operasi tertentu.

```
#include <stdio.h>

// Deklarasi fungsi
float multiply(float x, float y);

int main() {
    float result;
    result = multiply(4.5, 2.3);
    printf("Result: %.2f\n", result);
    return 0;
}
```

```
}  
  
// Definisi fungsi  
float multiply(float x, float y) {  
    return x * y;  
}
```

Penjelasan:

- `float multiply(float x, float y);` adalah deklarasi fungsi `multiply` yang menerima dua parameter `float` dan mengembalikan nilai `float`.
- `result = multiply(4.5, 2.3);` memanggil fungsi `multiply` dengan argumen 4.5 dan 2.3.
- Fungsi `multiply` dijelaskan setelah `main`, mengembalikan hasil perkalian `x` dan `y`.

2. Rekursi

Rekursi adalah teknik pemrograman di mana sebuah fungsi memanggil dirinya sendiri. Ini sering digunakan untuk memecahkan masalah yang dapat dipecah menjadi sub-masalah yang lebih kecil.

Rekursi Sederhana: Faktorial

Contoh klasik dari rekursi adalah menghitung faktorial dari sebuah angka.

```
#include <stdio.h>  
  
int factorial(int n);  
  
int main() {  
    int num = 5;  
    printf("Factorial of %d is %d\n", num, factorial(num));  
    return 0;  
}  
  
int factorial(int n) {  
    if (n == 0) {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}
```

Penjelasan:

- `int factorial(int n);` adalah deklarasi fungsi `factorial` yang menerima satu parameter `integer` dan mengembalikan nilai `integer`.
- `printf("Factorial of %d is %d\n", num, factorial(num));` memanggil fungsi `factorial` dengan argumen `num`.
- Fungsi `factorial` mengembalikan 1 jika `n` adalah 0. Jika tidak, fungsi mengembalikan `n * factorial(n - 1)`.

Rekursi Kompleks: Deret Fibonacci

Deret Fibonacci adalah contoh lain dari rekursi di mana setiap elemen dalam deret adalah jumlah dari dua elemen sebelumnya.

```
#include <stdio.h>

int fibonacci(int n);

int main() {
    int n = 10;
    printf("Fibonacci series up to %d terms:\n", n);
    for (int i = 0; i < n; i++) {
        printf("%d ", fibonacci(i));
    }
    printf("\n");
    return 0;
}

int fibonacci(int n) {
    if (n == 0) {
        return 0;
    } else if (n == 1) {
        return 1;
    } else {
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}
```

Penjelasan:

- `int fibonacci(int n);` adalah deklarasi fungsi fibonacci yang menerima satu parameter integer dan mengembalikan nilai integer.
- Fungsi fibonacci mengembalikan 0 jika `n` adalah 0, 1 jika `n` adalah 1, dan `fibonacci(n - 1) + fibonacci(n - 2)` jika `n` lebih besar dari 1.
- `printf("%d ", fibonacci(i));` memanggil fungsi fibonacci untuk setiap nilai `i` dari 0 hingga `n-1`.

Praktikum

Latihan 1: Fungsi untuk Menghitung Luas Lingkaran

Buatlah program yang memiliki fungsi untuk menghitung luas lingkaran berdasarkan radius yang diberikan.

```
#include <stdio.h>

#define PI 3.14159

// Deklarasi fungsi
float calculateArea(float radius);

int main() {
    float radius, area;
    printf("Enter the radius of the circle: ");
    scanf("%f", &radius);

    area = calculateArea(radius);
    printf("Area of the circle: %.2f\n", area);
    return 0;
}

// Definisi fungsi
float calculateArea(float radius) {
    return PI * radius * radius;
}
```

Penjelasan:

- `float calculateArea(float radius);` adalah deklarasi fungsi `calculateArea` yang menerima satu parameter float dan mengembalikan nilai float.
- `area = calculateArea(radius);` memanggil fungsi `calculateArea` dengan argumen `radius`.
- Fungsi `calculateArea` mengembalikan hasil perkalian `PI`, `radius`, dan `radius`.

Latihan 2: Rekursi untuk Menghitung Pangkat

Buatlah program yang memiliki fungsi rekursif untuk menghitung pangkat dari sebuah angka.

```
#include <stdio.h>

// Deklarasi fungsi
int power(int base, int exp);

int main() {
    int base, exp;
    printf("Enter base: ");
    scanf("%d", &base);
    printf("Enter exponent: ");
    scanf("%d", &exp);

    printf("%d^%d = %d\n", base, exp, power(base, exp));
    return 0;
}

// Definisi fungsi
int power(int base, int exp) {
    if (exp == 0) {
        return 1;
    } else {
        return base * power(base, exp - 1);
    }
}
```

Penjelasan:

- `int power(int base, int exp);` adalah deklarasi fungsi `power` yang menerima dua parameter integer dan mengembalikan nilai integer.
- `printf("%d^%d = %d\n", base, exp, power(base, exp));` memanggil fungsi `power` dengan argumen `base` dan `exp`.
- Fungsi `power` mengembalikan 1 jika `exp` adalah 0. Jika tidak, fungsi mengembalikan `base * power(base, exp - 1)`.

Latihan 3: Fungsi dan Rekursi untuk Menghitung Jumlah Digit

Buatlah program yang memiliki fungsi rekursif untuk menghitung jumlah digit dari sebuah angka.

```
#include <stdio.h>

// Deklarasi fungsi
int countDigits(int num);

int main() {
    int num;
```

```
printf("Enter a number: ");
scanf("%d", &num);

printf("Number of digits in %d is %d\n", num, countDigits(num));
return 0;
}

// Definisi fungsi
int countDigits(int num) {
    if (num == 0) {
        return 0;
    } else {
        return 1 + countDigits(num / 10);
    }
}
```

Penjelasan:

- `int countDigits(int num);` adalah deklarasi fungsi `countDigits` yang menerima satu parameter integer dan mengembalikan nilai integer.
- `printf("Number of digits in %d is %d\n", num, countDigits(num));` memanggil fungsi `countDigits` dengan argumen `num`.
- Fungsi `countDigits` mengembalikan 0 jika `num` adalah 0. Jika tidak, fungsi mengembalikan `1 + countDigits(num / 10)`.

LATIHAN MANDIRI

Latihan 1:

Buatlah program yang memiliki fungsi rekursif untuk membalikkan sebuah string.

```
#include <stdio.h>
#include <string.h>

// Deklarasi fungsi
void reverseString(char str[], int start, int end);

int main() {
    char str[100];
    printf("Enter a string: ");
    gets(str);

    reverseString(str, 0, strlen(str) - 1);
    printf("Reversed string: %s\n", str);
    return 0;
}

// Definisi fungsi
void reverseString(char str[], int start, int end) {
    if (start >= end) {
        return;
    }

    char temp = str[start];
    str[start] = str[end];
    str[end] = temp;

    reverseString(str, start + 1, end - 1);
}
```

Penjelasan:

- `void reverseString(char str[], int start, int end);` adalah deklarasi fungsi `reverseString` yang menerima tiga parameter: array karakter `str`, integer `start`, dan integer `end`.
- `reverseString(str, 0, strlen(str) - 1);` memanggil fungsi `reverseString` dengan argumen `str`, `0`, dan panjang string dikurangi 1.
- Fungsi `reverseString` menukar karakter pada posisi `start` dan `end`, kemudian memanggil dirinya sendiri dengan `start + 1` dan `end - 1` sampai `start` tidak lagi lebih kecil dari `end`.

Kuis 2:

Buatlah program yang memiliki fungsi rekursif untuk mencari elemen terbesar dalam sebuah array.

```
#include <stdio.h>

// Deklarasi fungsi
int findMax(int arr[], int n);

int main() {
    int n;
    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter the elements of the array:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("The maximum element in the array is: %d\n", findMax(arr, n));
    return 0;
}

// Definisi fungsi
int findMax(int arr[], int n) {
    if (n == 1) {
        return arr[0];
    }

    int max = findMax(arr, n - 1);
    if (arr[n - 1] > max) {
        return arr[n - 1];
    } else {
        return max;
    }
}
```

Penjelasan:

- `int findMax(int arr[], int n);` adalah deklarasi fungsi `findMax` yang menerima dua parameter: array integer `arr` dan integer `n`.
- `findMax(arr, n);` memanggil fungsi `findMax` dengan argumen `arr` dan `n`.
- Fungsi `findMax` mengembalikan elemen `arr[0]` jika `n` adalah 1. Jika tidak, fungsi mengembalikan elemen terbesar antara `arr[n-1]` dan elemen terbesar dari `n-1` elemen pertama array.

== Happy Coding :D ==